PUC-Rio | BI Master

TRABALHO DE CONCLUSÃO DE CURSO

Auditoria Inteligente de Dados com IA Generativa:

Detecção de Anomalias com LangChain e GPT

Orientador:
Leonardo Alfredo Forero Mendoza

Alunos:
João Gabriel Lopes Zarur
Thiago de Assis Sobreira

Rio de Janeiro — Março de 2026

SUMÁRIO

1. Introdução e Contexto
2. Objetivos do Trabalho
3. Arquitetura e Tecnologias Utilizadas
4. Parte 1 — Geração da Base de Dados Sintética
5. Parte 2 — Dashboard de Integridade de Dados
6. Parte 3 — Persistência em Banco de Dados SQLite
7. Parte 4 — Consulta SQL com Agente LLM (LangChain + GPT)
8. Parte 5 — Guardrail de Segurança SQL
9. Parte 6 — Análises em Linguagem Natural com o Agente
10. Resultados Obtidos
11. Conclusão

1. INTRODUÇÃO E CONTEXTO

A crescente digitalização dos processos educacionais gerou volumes expressivos de dados financeiros e acadêmicos, tornando crítica a necessidade de mecanismos robustos de auditoria e controle de qualidade.

Este Trabalho de Conclusão de Curso propõe uma solução inovadora que combina técnicas modernas de Engenharia de Dados com Inteligência Artificial Generativa para automatizar a detecção de anomalias em bases de dados de pagamentos. O sistema é capaz não apenas de identificar desvios estatísticos, mas também de responder a perguntas em linguagem natural sobre os dados, democratizando o acesso à auditoria.

Problema Central

Como identificar automaticamente inconsistências em grandes bases de dados de pagamentos e permitir que analistas façam perguntas sobre esses dados em linguagem natural, sem necessidade de conhecimento técnico em SQL?

2. OBJETIVOS DO TRABALHO

2.1 Objetivo Geral

Desenvolver um pipeline completo de auditoria inteligente que integre geração de dados sintéticos, detecção estatística de anomalias, persistência em banco de dados e consulta via agente de IA com linguagem natural.

2.2 Objetivos Específicos

- Gerar uma base de dados sintética que simule pagamentos reais com anomalias controladas (taxa configurável de 3% a 5%)
- Implementar um dashboard de integridade que exiba em tempo real os casos críticos de desvio
- Persistir os dados em banco SQLite para consultas estruturadas
- Integrar um agente SQL baseado em LLM (GPT-4.1-mini via LangChain) capaz de responder perguntas em português sobre os dados
- Implementar guardrail de segurança que bloqueie comandos SQL destrutivos
- Demonstrar o potencial da IA Generativa como ferramenta de auditoria acessível

3. ARQUITETURA E TECNOLOGIAS UTILIZADAS

3.1 Visão Geral da Arquitetura

O projeto foi estruturado em seis módulos principais, cada um responsável por uma etapa do pipeline de auditoria inteligente. A arquitetura segue o padrão de processamento em camadas, desde a geração dos dados até a interface em linguagem natural.

MÓDULO	FUNÇÃO	TECNOLOGIA
Parte 1	Geração de dados sintéticos com anomalias	Python / NumPy / Pandas
Parte 2	Dashboard de integridade visual no terminal	Python / ANSI Colors
Parte 3	Persistência em banco de dados relacional	SQLite / SQLAlchemy
Parte 4	Agente SQL com LLM para consultas naturais	LangChain / OpenAI GPT-4.1-mini
Parte 5	Guardrail de segurança para comandos SQL	Python / Regex
Parte 6	Análises em linguagem natural automatizadas	LangChain SQL Agent

3.2 Stack Tecnológica

- Python 3.x — linguagem principal do projeto
- Pandas e NumPy — manipulação e geração de dados
- SQLite — banco de dados relacional leve e embutido
- LangChain — framework de orquestração de LLMs
- OpenAI GPT-4.1-mini — modelo de linguagem para o agente SQL
- SQLAlchemy — ORM para abstração do banco de dados

4. PARTE 1 — GERAÇÃO DA BASE DE DADOS SINTÉTICA

O primeiro módulo do sistema é responsável por gerar uma base de dados realista que simula o comportamento dos pagamentos ao longo de dois dias consecutivos. A função gerar_dados_dashboard() permite configurar o volume e a taxa de anomalias desejada.

4.1 Lógica de Geração

- São criados registros para dois dias: 2025-05-29 (dia base) e 2025-05-30 (dia de comparação)
- Cada aluno possui três valores financeiros: boleto, receita e bolsa
- A taxa de anomalia (padrão: 3%) determina quantos alunos terão variações fora do padrão
- Alunos normais variam ±1% entre os dias; alunos anômalos variam ±20% a ±30%

4.2 Código — Geração dos Dados

```
def gerar_dados_dashboard(total_alunos=1000, taxa_anomalia=0.05):
    np.random.seed(42)
    alunos_ids = [f'MATRICULA_{i:04d}' for i in range(1, total_alunos + 1)]
    qtd_com_erro = int(total_alunos * taxa_anomalia)
    alunos_com_erro = np.random.choice(alunos_ids, size=qtd_com_erro,
replace=False)
    # Fator de variação anômala: 0.80, 1.25, 0.70 ou 1.30
    fator = np.random.choice([0.80, 1.25, 0.70, 1.30])
```

Parâmetros Utilizados na Execução

base_auditoria = gerar_dados_dashboard(total_alunos=1000, taxa_anomalia=0.03) → 1.000 alunos monitorados | ~30 alunos com anomalias geradas

4.3 Output — Visualização dos Dados Gerados

A seguir, o print da saída do comando base_auditoria.tail(), mostrando a estrutura dos dados gerados:

```
...
    ==============================================================================
                DASHBOARD DE INTEGRIDADE DE DADOS - PROCESSAMENTO AI
    ==============================================================================
     STATUS DO BANCO: CRÍTICO
     TOTAL DE ALUNOS MONITORADOS: 1000
     ANOMALIAS DETECTADAS: 30
    ------------------------------------------------------------------------------
    TOP 10 CASOS PARA INTERVENÇÃO IMEDIATA (Ordenado por Gravidade):
    ID ALUNO        | VAR BOLETO   | VAR ROB      | GRAVIDADE
    ------------------------------------------------------------------------------
    MATRICULA_0903  |      -30.0% |      -30.0% | ALTA
    MATRICULA_0102  |       30.0% |       30.0% | ALTA
    MATRICULA_0236  |      -30.0% |      -30.0% | ALTA
    MATRICULA_0528  |      -30.0% |      -30.0% | ALTA
    MATRICULA_0372  |       30.0% |       30.0% | ALTA
    MATRICULA_0175  |      -30.0% |      -30.0% | ALTA
    MATRICULA_0522  |      -30.0% |      -30.0% | ALTA
    MATRICULA_0860  |       30.0% |       30.0% | ALTA
    MATRICULA_0884  |       30.0% |       30.0% | ALTA
    MATRICULA_0812  |      -30.0% |      -30.0% | ALTA

    ... e mais 20 registros ocultos. Verifique o log completo.
    ==============================================================================

    Visualização das colunas processadas (Head):
            id_aluno        data       boleto          rob       bolsa
    1995  MATRICULA_0998  2025-05-30  1285.807423  1903.538768  191.615827
    1996  MATRICULA_0999  2025-05-29   836.556424  1254.834635  125.483464
    1997  MATRICULA_0999  2025-05-30   831.611511  1250.829493  125.483464
    1998  MATRICULA_1000  2025-05-29  1412.696909  2119.045363  211.904536
    1999  MATRICULA_1000  2025-05-30  1415.949761  2136.447206  211.904536
```

5. PARTE 2 — DASHBOARD DE INTEGRIDADE DE DADOS

O segundo módulo implementa um dashboard visual no terminal que apresenta ao analista de dados um panorama completo da saúde dos dados processados. O sistema calcula automaticamente as variações percentuais entre os dias e classifica as anomalias por gravidade.

5.1 Lógica de Detecção

- Calcula variação percentual de boleto e ROB para cada aluno entre os dois dias
- Classifica como anomalia qualquer desvio superior ao limiar configurável (padrão: 10%)
- Ordena as anomalias por gravidade decrescente (maior desvio primeiro)
- Classifica como ALTA gravidade quando o desvio ultrapassa 15%

5.2 Código — Função de Dashboard

```
def exibir_dashboard(df, data_hoje='2025-05-30', limite_top=10,
limiar_anomalia=10):
    df['dif_perc_boleto'] = df.groupby('id_aluno')['boleto'].pct_change() * 100
    df['dif_perc_rob']    = df.groupby('id_aluno')['rob'].pct_change() * 100
    anomalias = hoje[(hoje['dif_perc_boleto'].abs() > limiar_anomalia) |
                     (hoje['dif_perc_rob'].abs() > limiar_anomalia)]
```

5.3 Output — Dashboard no Terminal

A seguir, o print do dashboard gerado no terminal, mostrando os top 10 casos de anomalia com suas respectivas variações percentuais e níveis de gravidade:

```
print(df_top10)

...
      id_aluno        data       boleto          rob  dif_perc_boleto  dif_perc_rob  maior_desvio
1805  MATRICULA_0903  2025-05-30  1212.504006  1818.756009            -30.0         -30.0          30.0
203   MATRICULA_0102  2025-05-30  2034.081264  3051.121897             30.0          30.0          30.0
471   MATRICULA_0236  2025-05-30  1068.652052  1602.978079            -30.0         -30.0          30.0
1055  MATRICULA_0528  2025-05-30   759.444113  1139.166170            -30.0         -30.0          30.0
743   MATRICULA_0372  2025-05-30  2071.738153  3107.607230             30.0          30.0          30.0
349   MATRICULA_0175  2025-05-30  1174.266856  1761.400284            -30.0         -30.0          30.0
1043  MATRICULA_0522  2025-05-30   587.422369   881.133554            -30.0         -30.0          30.0
1719  MATRICULA_0860  2025-05-30  2025.013517  3037.520276             30.0          30.0          30.0
1767  MATRICULA_0884  2025-05-30  1201.738586  1802.607879             30.0          30.0          30.0
1623  MATRICULA_0812  2025-05-30   928.243383  1392.365075            -30.0         -30.0          30.0
```

6. PARTE 3 — PERSISTÊNCIA EM BANCO DE DADOS SQLITE

Após a geração e análise inicial dos dados, o terceiro módulo persiste o DataFrame em um banco de dados SQLite. Esta etapa é fundamental para permitir que o agente de IA realize consultas SQL dinâmicas sobre os dados auditados.

6.1 Gravação dos Dados

```
DB_PATH = 'auditoria.db'
TABLE_NAME = 'auditoria_pagamentos'
conn = sqlite3.connect(DB_PATH)
base_auditoria.to_sql(TABLE_NAME, conn, if_exists='replace', index=False)
```

6.2 Schema da Tabela

Coluna	Tipo	Descrição
id_aluno	**TEXT**	Identificador único do aluno (ex: MATRICULA_0001)
data	**TEXT**	Data do registro (2025-05-29 ou 2025-05-30)
boleto	**REAL**	Valor do boleto do aluno no respectivo dia
rob	**REAL**	Resultado Operacional Bruto do aluno
bolsa	**REAL**	Valor da bolsa associada ao aluno

6.3 Output — Confirmação do Banco de Dados

Print da saída confirmando a criação do banco, total de registros e schema da tabela:

```
conn.close()

Total de registros na tabela: 2000
Schema (colunas):
(0, 'id_aluno', 'TEXT', 0, None, 0)
(1, 'data', 'TEXT', 0, None, 0)
(2, 'boleto', 'REAL', 0, None, 0)
(3, 'rob', 'REAL', 0, None, 0)
(4, 'bolsa', 'REAL', 0, None, 0)
```

7. PARTE 4 — CONSULTA SQL COM AGENTE LLM (LANGCHAIN + GPT)

O quarto módulo representa o núcleo de inovação do trabalho: a integração de um agente de IA baseado em LLM (Large Language Model) capaz de receber perguntas em linguagem natural, converter automaticamente para SQL, executar no banco e retornar os resultados interpretados.

7.1 Configuração do Agente

```
from langchain_community.utilities import SQLDatabase
from langchain_community.agent_toolkits import create_sql_agent
from langchain_openai import ChatOpenAI

db  = SQLDatabase.from_uri(f'sqlite:///{DB_PATH}')
llm = ChatOpenAI(model='gpt-4.1-mini', temperature=0)
agent = create_sql_agent(llm=llm, db=db, verbose=True)
```

> **Por que temperature=0?**
> O parâmetro temperature=0 garante que o modelo seja determinístico, ou seja, produza sempre a mesma resposta para a mesma pergunta. Em tarefas de auditoria de dados, a reprodutibilidade é essencial.

7.2 Primeira Consulta — Contagem de Registros

A primeira consulta ao agente valida a integração: o modelo converte automaticamente a pergunta em linguagem natural para SQL e retorna o resultado.

Pergunta: "Quantos registros existem na tabela auditoria_pagamentos?"

```
agent.invoke({"input": f"Quantos registros existem na tabela {TABLE_NAME}?"})

> Entering new SQL Agent Executor chain...
Action: sql_db_list_tables
Action Input:
auditoria_pagamentosQuestion: Quantos registros existem na tabela auditoria_pagamentos?
Thought: I know the table name is auditoria_pagamentos. I should count the number of records in this table.
Action: sql_db_query_checker
Action Input: SELECT COUNT(*) FROM auditoria_pagamentos;SELECT COUNT(*) FROM auditoria_pagamentos;Action: sql_db_qu
Action Input: SELECT COUNT(*) FROM auditoria_pagamentos;[(2000,)]Final Answer: A tabela auditoria_pagamentos possui

> Finished chain.
{'input': 'Quantos registros existem na tabela auditoria_pagamentos?',
 'output': 'A tabela auditoria_pagamentos possui 2000 registros.'}
```

7.3 Consulta Avançada — Top 10 Anomalias

A segunda consulta demonstra a capacidade analítica do agente: calcular variações percentuais via SQL e retornar os casos mais críticos.

```
LIMIT 10;
[('MATRICULA_0903', -30.000000000000014, -30.000000000000004, 30.0000000
| id_aluno       | dif_perc_boleto | dif_perc_rob | maior_desvio |
|----------------|-----------------|--------------|--------------|
| MATRICULA_0903 | -30.00          | -30.00       | 30.00        |
| MATRICULA_0102 | 30.00           | 30.00        | 30.00        |
| MATRICULA_0175 | -30.00          | -30.00       | 30.00        |
| MATRICULA_0236 | -30.00          | -30.00       | 30.00        |
| MATRICULA_0281 | -30.00          | -30.00       | 30.00        |
| MATRICULA_0372 | 30.00           | 30.00        | 30.00        |
| MATRICULA_0522 | -30.00          | -30.00       | 30.00        |
| MATRICULA_0528 | -30.00          | -30.00       | 30.00        |
| MATRICULA_0637 | -30.00          | -30.00       | 30.00        |
| MATRICULA_0679 | 30.00           | 30.00        | 30.00        |

These are the 10 students with the largest absolute percentage deviatio
```

8. PARTE 5 — GUARDRAIL DE SEGURANÇA SQL

O quinto módulo implementa uma camada essencial de segurança: um mecanismo de validação que impede a execução de comandos SQL destrutivos, garantindo que o agente de IA opere exclusivamente com operações de leitura (SELECT).

8.1 Motivação

Sistemas de IA Generativa que executam SQL automaticamente apresentam riscos de segurança caso não haja controle sobre os tipos de operação permitidos. Um agente mal-intencionado ou uma instrução ambígua poderia resultar em DELETE, DROP TABLE ou outras operações irreversíveis.

8.2 Implementação do Guardrail

```
def somente_select(sql: str) -> bool:
    s = sql.strip().lower()
    proibidas = ['insert', 'update', 'delete', 'drop',
                 'alter', 'create', 'truncate', 'attach', 'pragma']
    if any(p in s for p in proibidas):
        return False
    return s.startswith('select') or s.startswith('with')
```

8.3 Testes de Validação

Comando SQL	Resultado	Status
`SELECT * FROM auditoria_pagamentos LIMIT 5;`	True	PERMITIDO
`DROP TABLE auditoria_pagamentos;`	False	BLOQUEADO

```
import re

def somente_select(sql: str) -> bool:

    s = sql.strip().lower()

    proibidas = ["insert", "update", "delete", "drop", "alter", "create", "truncate", "attach", "pragma"]
    if any(p in s for p in proibidas):
        return False

    return s.startswith("select") or s.startswith("with")


teste_sql_ok = "SELECT * FROM auditoria_pagamentos LIMIT 5;"
teste_sql_bad = "DROP TABLE auditoria_pagamentos;"

print("OK?", somente_select(teste_sql_ok))
print("OK?", somente_select(teste_sql_bad))
```

```
OK? True
OK? False
```

9. PARTE 6 — ANÁLISES EM LINGUAGEM NATURAL COM O AGENTE

O sexto e último módulo demonstra o poder do agente de IA em responder perguntas analíticas complexas formuladas inteiramente em linguagem natural. O sistema executa automaticamente cinco análises distintas sobre a base de auditoria.

9.1 Perguntas Executadas

Todas as perguntas incluem o contexto de cálculo: 'Calcule a variação percentual comparando os valores do dia 2025-05-30 com o dia 2025-05-29.'

Pergunta 1

Quantas anomalias (variação de boleto ou rob > 10%) ocorreram?

```
        JOIN auditoria_pagamentos b ON a.id_aluno = b.id_aluno
        WHERE a.data = '2025-05-29' AND b.data = '2025-05-30'
    ) variations
    WHERE boleto_variation > 10 OR rob_variation > 10;
    [(30,)]Final Answer: Ocorreu um total de 30 anomalias com variação percentual maior que 10% no boleto ou no rob entre os dias 2025-05-29 e 2025-05-30 na tabela auditoria_pagamentos.

    > Finished chain.
    Ocorreu um total de 30 anomalias com variação percentual maior que 10% no boleto ou no rob entre os dias 2025-05-29 e 2025-05-30 na tabela auditoria_pagamentos.
```

Pergunta 2

Qual a média e o máximo da variação absoluta de boleto e rob no dia 2025-05-30?

```
... LIMIT 10;
    ```sql
    SELECT
      a1.id_aluno,
      ((a1.boleto - a2.boleto) / a2.boleto) * 100 AS perc_var_boleto,
      ((a1.rob - a2.rob) / a2.rob) * 100 AS perc_var_rob,
      ABS(a1.boleto - a2.boleto) AS abs_var_boleto,
      ABS(a1.rob - a2.rob) AS abs_var_rob
    FROM auditoria_pagamentos a1
    JOIN auditoria_pagamentos a2 ON a1.id_aluno = a2.id_aluno
    WHERE a1.data = '2025-05-30' AND a2.data = '2025-05-29'
    LIMIT 10;
    ```Action: sql_db_query
    Action Input:
    SELECT
      AVG(ABS(a1.boleto - a2.boleto)) AS avg_abs_var_boleto,
      MAX(ABS(a1.boleto - a2.boleto)) AS max_abs_var_boleto,
      AVG(ABS(a1.rob - a2.rob)) AS avg_abs_var_rob,
      MAX(ABS(a1.rob - a2.rob)) AS max_abs_var_rob
    FROM auditoria_pagamentos a1
    JOIN auditoria_pagamentos a2 ON a1.id_aluno = a2.id_aluno
    WHERE a1.data = '2025-05-30' AND a2.data = '2025-05-29';[(17.819101240351472, 595.867793278361, 26.350487912765541, 893.80168991754 14)]Final Answer: A média da variação absoluta de boleto no dia 2025-05-30 em relação a 2025-05-29 é aproximadamente 17.82, e o máximo da variação absoluta de boleto é aproxim

    > Finished chain.
    A média da variação absoluta de boleto no dia 2025-05-30 em relação a 2025-05-29 é aproximadamente 17.82, e o máximo da variação absoluta de boleto é aproximadamente 595.87. Para rob, a média da variação absoluta é aproximadamente 26.35, e o máximo da variação absoluta é aproximadamente 893.80.
```

Pergunta 3

Quais alunos tiveram desvio acima de 15% (crítico)? Liste os 20 maiores.

```
> Finished chain.
Os 20 maiores IDs de alunos que tiveram variação percentual crítica (acima de 15%) no boleto ou rob entre os dias 2025-05-29 e 2025-05-30 são:

| id_aluno       | boleto_variation (%) | rob_variation (%) |
|----------------|----------------------|-------------------|
| MATRICULA_0987 | -30.00               | -30.00            |
| MATRICULA_0974 | -30.00               | -30.00            |
| MATRICULA_0948 | -20.00               | -20.00            |
| MATRICULA_0939 | -20.00               | -20.00            |
| MATRICULA_0903 | -30.00               | -30.00            |
| MATRICULA_0900 | -20.00               | -20.00            |
| MATRICULA_0884 | 30.00                | 30.00             |
| MATRICULA_0860 | 30.00                | 30.00             |
| MATRICULA_0812 | -30.00               | -30.00            |
| MATRICULA_0762 | -30.00               | -30.00            |
| MATRICULA_0741 | 30.00                | 30.00             |
| MATRICULA_0738 | -20.00               | -20.00            |
| MATRICULA_0679 | 30.00                | 30.00             |
| MATRICULA_0661 | -20.00               | -20.00            |
| MATRICULA_0637 | -30.00               | -30.00            |
| MATRICULA_0627 | 25.00                | 25.00             |
| MATRICULA_0550 | 25.00                | 25.00             |
| MATRICULA_0528 | -30.00               | -30.00            |
| MATRICULA_0522 | -30.00               | -30.00            |
| MATRICULA_0514 | 25.00                | 25.00             |
```

Pergunta 4

Entre a variação de boleto e a de rob, qual teve mais ocorrências acima de 10%?

```
    Action Input:
    SELECT
      SUM(CASE WHEN ABS(((b.boleto - a.boleto) / a.boleto) * 100) > 10 THEN 1 ELSE 0 END) AS count_boleto_above_10,
      SUM(CASE WHEN ABS(((b.rob - a.rob) / a.rob) * 100) > 10 THEN 1 ELSE 0 END) AS count_rob_above_10
    FROM auditoria_pagamentos a
    JOIN auditoria_pagamentos b ON a.id_aluno = b.id_aluno
    WHERE a.data = '2025-05-29' AND b.data = '2025-05-30';
    [(30, 30)]Thought: Both boleto and rob have 30 occurrences of percentage variation above 10%, so they are tied in the number of occurrences above 10%.

    Final Answer: Tanto a variação percentual de boleto quanto a de rob tiveram 30 ocorrências acima de 10%, ou seja, empataram.

    > Finished chain.
    Tanto a variação percentual de boleto quanto a de rob tiveram 30 ocorrências acima de 10%, ou seja, empataram.
```

10. RESULTADOS OBTIDOS

O sistema desenvolvido demonstrou capacidade robusta de detecção e análise de anomalias. A seguir, um resumo consolidado dos resultados obtidos durante a execução do pipeline completo.

MÉTRICA	RESULTADO
Total de alunos monitorados	1.000
Taxa de anomalias configurada	3% (30 alunos)
Limiar de detecção	> 10% de variação
Perguntas analíticas respondidas pelo agente	7 (incluindo as 5 automatizadas)
Comandos SQL gerados automaticamente pelo LLM	Variável por pergunta (verbose=True)
Guardrail: comandos destrutivos bloqueados	100% dos testes

11. CONCLUSÃO

Este trabalho demonstrou que a combinação de técnicas de Engenharia de Dados com IA Generativa representa uma abordagem poderosa e acessível para problemas de auditoria de dados em larga escala. O pipeline desenvolvido é capaz de processar bases de dados de milhares de registros, detectar automaticamente anomalias estatísticas e responder a perguntas complexas formuladas em linguagem natural.

A integração do framework LangChain com o modelo GPT-4.1-mini da OpenAI mostrou-se eficiente para a geração automática de consultas SQL, eliminando a necessidade de conhecimento técnico especializado por parte do analista. O agente é capaz de interpretar a intenção da pergunta, construir a query adequada e apresentar os resultados de forma clara e interpretável.

11.1 Contribuições do Trabalho

- Framework end-to-end de auditoria de dados com IA Generativa
- Demonstração prática da capacidade analítica de LLMs em domínios de dados financeiros
- Implementação de guardrail de segurança para sistemas de agentes SQL
- Base sintética configurável para testes de sistemas de detecção de anomalias

11.2 Trabalhos Futuros

- Integração com fontes de dados reais e pipelines de ETL em produção
- Expansão do guardrail com análise semântica mais sofisticada
- Interface web para democratizar o acesso ao sistema de auditoria
- Avaliação de outros modelos LLM (Claude, Gemini) para comparação de desempenho
- Implementação de alertas automáticos por e-mail ou Slack em caso de anomalias

Considerações Finais

O projeto evidencia que ferramentas de IA Generativa, quando corretamente integradas a pipelines de dados, têm potencial transformador para áreas de compliance, auditoria e governança de dados — tornando essas funções mais ágeis, acessíveis e robustas.

REFERÊNCIAS

CHASE, H.; et al. LangChain: Building applications with LLMs through composability. GitHub, 2023. Disponível em: https://github.com/langchain-ai/langchain

OPENAI. GPT-4 Technical Report. arXiv, 2023. Disponível em: https://arxiv.org/abs/2303.08774

PANDAS DEVELOPMENT TEAM. pandas: powerful Python data analysis toolkit. Disponível em: https://pandas.pydata.org

SQLITE CONSORTIUM. SQLite Documentation. Disponível em: https://www.sqlite.org/docs.html

MCKINNEY, W. Python for Data Analysis. 3. ed. O'Reilly Media, 2022.